Learning from data: tasks, representations and notation

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Machine learning replaces hand-written rules by functions estimated from examples. This note fixes the vocabulary and notation of the course: what a dataset is, how an image, a text or a collision event becomes a vector, which learning problems exist, what the ideal predictor for each is and how much error no predictor can avoid. It also sets the conventions for running times and the vector toolkit (norms, inner products, cosine similarity, pairwise distances) that every later note uses. Later notes refer back to Section 2 (notation) and Table 2 (tasks and losses) instead of repeating them.

1 Learning from examples

Suppose we want a program that decides whether a shower in a calorimeter was made by an electron or by a charged pion. Writing the rules by hand — electrons deposit their energy early and in a narrow core, pions penetrate deeper and spread wider, unless the pion transferred most of its energy to a neutral pion in its first interaction — quickly turns into a long list of exceptions. The machine-learning approach is different: simulate or record many showers whose origin is known and let an algorithm *find* a function that maps the cell energies to the label. Nothing in that procedure is specific to calorimeters: fed with other labelled examples, the same algorithm sorts galaxy images by shape, transcribes speech or estimates the energy of the shower itself.

> **Key idea** Fitting the training set is easy; predicting new cases is the point. A table that stores every training shower with its label answers perfectly on those showers and is useless on a new one. The property we are after is *generalisation*: the quality of the predictions on inputs not seen during training. Note 4 shows how to measure it honestly.

How hopeless is memorising? Black-and-white 3×3 images come in only $2^9 = 512$ versions, but 28×28 digits (Table 1), even with pixels reduced to on or off, come in $2^{784} \approx 10^{236}$; 60 000 training images cover a vanishing fraction, and a new digit is essentially never an exact copy of one. Every useful prediction is an *inference* from similar seen inputs, and the course is about what "similar" should mean.

1.1 Training and prediction

A learning algorithm consists of two procedures. Train turns the training set into a *model* — a list of numbers, a tree, the training set itself; Predict turns the model and a new input $\mathbf{x}$ into a prediction. Their costs are counted separately, because a model is trained once and queried many times. Algorithm 1 is the memoriser of the key idea above, written in this form; when the query has not been seen it falls back on the most frequent label, the simplest *baseline* that every real method must beat. Here n is the number of training examples, d the number of real numbers that describe

one input and m the number of classes (Section 2); a *hash table*, like a Python `dict`, finds the value stored under a key in a time that does not grow with the number of keys. Section 2.1 explains the "Running time" boxes and the $O(\cdot)$ notation.

Algorithm 1 The memoriser: a lookup table with a majority-class fallback

Require: training set $D_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ with $\mathbf{x}_i \in \mathbb{R}^d$, labels in $\{1, \dots, m\}$

function TRAIN(D_n)
    $T \leftarrow$ hash table with $T[\mathbf{x}_i] = y_i$ for $i = 1, \dots, n$ ▷ key: input vector, value: label
    $c \leftarrow$ most frequent label among $y_1, \dots, y_n$; **return** model (T, c)
end function
function PREDICT((T, c), $\mathbf{x}$)
    return $T[\mathbf{x}]$ if $\mathbf{x}$ is a key of T, otherwise c
end function

Running time — the memoriser (Algorithm 1)

Training $O(nd + m)$: hashing each of the n keys reads its d entries; counting labels needs m counters. **Prediction** $O(d)$ per query on average: hash the query and compare it with the stored key. **Memory** $O(nd)$: the whole training set. Speed is never the obstacle here; generalisation is.

From lookup to neighbours. The memoriser fails because an exact match almost never occurs. The smallest repair is to answer with the label of the *closest* stored example in Euclidean distance (Section 6): the *nearest-neighbour* or 1-NN rule. Letting the k closest examples vote and predicting the majority label gives the k-NN rule. Both are developed, with pseudocode and costs, in Note 5, Sections 1–2; Section 3 of this note tries them on small images.

Modern connection — rules versus data

A large language model is given no grammar: a transformer network [15] is trained to predict the next piece of text on hundreds of billions of tokens or more, and grammar, facts and the ability to solve new tasks from a few examples emerge as by-products [2]. Classifiers trained on the 1.28 million images of the ImageNet challenge [14] replaced decades of hand-designed vision pipelines.

In physics — from cuts to classifiers

Analyses traditionally separated signal from background with hand-tuned *cuts* ("transverse momentum above 25 GeV, invariant mass within 10 GeV of the Z peak"). Today a classifier trained on simulated events often makes the decision; for boosted top quarks, taggers reading low-level detector information beat the best hand-designed variables [9]. The physics moves into the simulation, the inputs and the validation.

2 Data and notation

Examples. An *example* (observation, sample) is a pair $(\mathbf{x}, y)$. The *input* $\mathbf{x}$ is a vector of d real numbers, $\mathbf{x} = (x_1, \dots, x_d)^{\mathsf{T}} \in \mathbb{R}^d$, whose components are called *features*. The *target* (label, desired output) y is what we want to predict. A *training set* of n examples is

$$D_n = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}.$$

Stacking the inputs as rows gives the *design matrix* $X \in \mathbb{R}^{n\times d}$ whose i-th row is $\mathbf{x}_i^{\mathsf{T}}$; the targets form a vector $\mathbf{y} \in \mathbb{R}^n$ (or, for m classes, a matrix of one-hot rows, Section 3.1). Indices follow one rule: a bold letter with one subscript, $\mathbf{x}_i$, is example i; a plain letter with one subscript, x_j or a_j, is component j of the vector $\mathbf{x}$ or $\mathbf{a}$; two indices, $x_{i,j}$ or $\mathbf{x}_i[j]$, give feature j of example i. A *predictor* is a function $f : \mathbb{R}^d \to \mathcal{Y}$, where $\mathcal{Y}$ is the set of possible targets (a set of classes, or $\mathbb{R}$); a new input whose target is unknown is a *test point*. The *indicator* $\mathbf{1}\{A\}$ equals 1 if the statement A is true and 0 otherwise.

Worked example — a dataset as a matrix

A trigger study records, for each collision event, the number of reconstructed tracks, the total energy deposited in the calorimeter and the missing transverse energy (both in GeV), and labels the event as signal ($y = +1$) or background ($y = -1$). Four invented events:

event i	tracks $x_{i,1}$	calorimeter energy $x_{i,2}$	missing E_{T} $x_{i,3}$	y_i
1	12	85.0	4.1	-1
2	3	41.5	38.2	$+1$
3	25	140.2	7.7	-1
4	5	62.0	51.3	$+1$

Here $n = 4$, $d = 3$, the design matrix X is the 4×3 block left of the bar and $\mathbf{y} = (-1, +1, -1, +1)^{\mathsf{T}}$. Row 3 is the input $\mathbf{x}_3 = (25, 140.2, 7.7)^{\mathsf{T}}$, the entry $x_{3,2} = \mathbf{x}_3[2] = 140.2$ GeV is feature 2 of example 3. The columns have different units and spreads, which matters for distances (Note 5, Section 5). On disk or in a data frame the whole dataset is usually one $n\times(d+1)$ array $[X \mid \mathbf{y}]$ — here the 4×4 table itself — or an $n\times(d+m)$ array when the targets are one-hot rows (Section 3.1).

Problem size. Three numbers decide which algorithms are feasible: n, d and the number of outputs (for classification, the number of classes m); Table 1 gives orders of magnitude.

Table 1. Typical problem sizes. The cost of most algorithms grows with n, d and m in very different ways.

	n (examples)	d (input dimension)	m (classes)
Iris flowers (1936)	150	4	3
Handwritten digits (MNIST)	60 000	784 (28×28 pixels)	10
ImageNet classification	1.28×10^6	150 528 (224×224×3)	1000
Language-model pre-training	10^{12}–10^{13} tokens	embedding 10^3–10^4	vocabulary 10^4–10^5
Jet tagging at the LHC	10^6–10^8 simulated jets	10^2–10^3 constituents' features	2–10

The i.i.d. assumption. We model each example $(\mathbf{X}_i, Y_i)$ as a draw from an *unknown* joint distribution $P(\mathbf{X}, Y)$, fixed by the physics and by the measuring apparatus. Capital letters denote random variables and lowercase letters the values they take: $\mathbf{X}$ (a plain X when the input is a single number) and Y versus $\mathbf{x}$ and y. The design matrix X and the one-hot target matrix Y of Section 3.1 are matrices, not random variables; the context always tells which is meant. Unless stated otherwise the examples are assumed *independent and identically distributed* (i.i.d.): every example comes from the same distribution, independently of the others. Two consequences are used constantly: a learning algorithm may visit the examples in any order without changing what it can learn, and a randomly chosen subset is a fair sample of the whole.

Datasets that break the i.i.d. assumption Time series (tomorrow depends on today), grouped data (several images of the same galaxy or patient), data whose distribution changes over time, and models trained on simulation but applied to real detector data (the *simulation-to-reality gap*) all violate the assumption. Ignoring this gives optimistic results (Section 5.1); Note 4 shows how to split such data.

Geometric picture. A dataset is a cloud of n labelled points in $\mathbb{R}^d$; learning means deciding what to predict *everywhere* in $\mathbb{R}^d$, including where no training point lies. Its difficulties in large d are the subject of Note 3.

2.1 Counting the cost of an algorithm

Every algorithm in these notes comes with a box "Running time" that states three costs: **training time**, **prediction time per query** and **memory**. They are given in $O(\cdot)$ notation: a cost $O(nd)$ means that, for large n and d, the number of elementary operations is at most a constant times nd. Constants and lower-order terms are dropped, because what decides feasibility is how the cost *grows*: doubling n doubles an $O(n)$ cost, quadruples an $O(n^2)$ one and multiplies an $O(d^3)$ cost by eight when d doubles. Besides n, d and m, the symbol q counts queries processed together; each note defines any further symbol it needs.

Operation	Cost	Why
inner product $\mathbf{a}^\mathsf{T}\mathbf{b}$, $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$	$O(d)$	d products and $d-1$ sums
matrix–vector product $X\mathbf{w}$, $X \in \mathbb{R}^{n\times d}$	$O(nd)$	n inner products of length d
matrix product QX^T, $Q \in \mathbb{R}^{q\times d}$	$O(qnd)$	qn inner products of length d
solving a $d \times d$ linear system	$O(d^3)$	Gaussian elimination
sorting n numbers	$O(n \log n)$	comparison sorting
the k smallest of n numbers ($k \le n$)	$O(n)$	partial selection, no full sort
insert or look up a key in a hash table	$O(1)$ on average	plus the cost of hashing the key

Memory. A real number takes 8 bytes in `float64` and 4 in `float32`, the GPU default. The MNIST design matrix, $60\,000 \times 784$ numbers, needs 188 MB in single precision; the ImageNet training images, resized to 224×224 at one byte per pixel and colour channel, need about 193 GB, more than most computers' memory, so they are streamed from disk in mini-batches (Note 7).

What vectorisation and GPUs change. Two algorithms with the same $O(\cdot)$ cost can differ by a factor of a hundred or more: an interpreted loop pays an overhead per elementary operation, while one matrix product runs in optimised parallel code on a CPU or GPU (Section 6.1). Hardware changes the constant, never the growth: an $O(n^2)$ algorithm with $n = 10^9$ needs 10^{18} operations, about four months on a workstation sustaining 10^{11} operations per second.

3 From raw observations to vectors

Most learning algorithms of this course require each example to be a vector of fixed dimension d. How we build that vector — the *representation* — often matters more than the choice of algorithm. Several recipes follow; most are worked by hand on a small object.

Raw representations. A 28×28 grey-scale image becomes a vector of $d = 784$ intensities; a waveform or a detector read-out can be used as is. Raw vectors are easy to produce but awkward: shifting the object by one pixel changes almost every coordinate.

Worked example — a 3×3 image, raw and with features

Three binary images, with dark pixels equal to 1:

$A =$  $B =$  $C =$ 

A is a vertical bar, B the same bar moved one pixel to the right and C a horizontal bar. Reading the pixels row by row ("row-major" order, the convention of `numpy`'s `reshape`) turns each image into a vector in $\mathbb{R}^9$:

$$\mathbf{x}_A = (0,1,0,\ 0,1,0,\ 0,1,0), \quad \mathbf{x}_B = (0,0,1,\ 0,0,1,\ 0,0,1), \quad \mathbf{x}_C = (0,0,0,\ 1,1,1,\ 0,0,0).$$

The squared Euclidean distance counts the pixels in which two images differ: $\|\mathbf{x}_A - \mathbf{x}_B\|^2 = 6$ (no dark pixel in common), but $\|\mathbf{x}_A - \mathbf{x}_C\|^2 = 4$ (they share the centre pixel). In the raw representation the *shifted copy* of the vertical bar is farther from it than a bar of a *different orientation.*

Now describe each image by three hand-crafted features: the number of dark pixels, the height and the width of the smallest box containing them. Then $\mathbf{x}'_A = \mathbf{x}'_B = (3,3,1)$ and $\mathbf{x}'_C = (3,1,3)$, so $\|\mathbf{x}'_A - \mathbf{x}'_B\|^2 = 0$ and $\|\mathbf{x}'_A - \mathbf{x}'_C\|^2 = 8$. The features are invariant to shifts and separate the two orientations, at the price of forgetting where the bar is — a good trade if orientation is what we want to predict, a bad one if position matters.

Three predictors on the unseen image. Train on $A \mapsto$ "vertical" and $C \mapsto$ "horizontal" and query B. The memoriser (Algorithm 1) finds no key equal to $\mathbf{x}_B$ and falls back on the majority label; with one example per class that is a tie, settled by an arbitrary rule (say, the first label alphabetically), so its answer is a guess. The 1-NN rule in raw pixels compares $\|\mathbf{x}_B - \mathbf{x}_A\|^2 = 6$ with $\|\mathbf{x}_B - \mathbf{x}_C\|^2 = 4$ and answers "horizontal", wrongly. On the features, $\|\mathbf{x}'_B - \mathbf{x}'_A\|^2 = 0 < \|\mathbf{x}'_B - \mathbf{x}'_C\|^2 = 8$, and the same rule answers "vertical". Same algorithm, better representation, correct answer.

Hand-crafted features. Experts design informative quantities: in vision, descriptors of local edge directions [4, 10]; in physics, invariant masses, transverse momenta, jet substructure.

3.1 Categorical attributes: one-hot encoding

Many attributes take one of K unordered values: the type of a reconstructed particle (electron, muon, photon, hadron), the detector sub-system that fired, the filter through which a galaxy was observed. The tempting code "electron = 1, muon = 2, photon = 3, hadron = 4" invents an order and distances that do not exist: a hadron would be three times as far from an electron as a muon is.

Definition 1 (one-hot encoding). Number the K categories $1, \dots, K$. The one-hot code of category k is the k-th standard basis vector $\mathbf{e}_k \in \{0,1\}^K$: a vector of zeros with a single one in position k.

With the particle types, electron $\mapsto (1,0,0,0)$, muon $\mapsto (0,1,0,0)$, photon $\mapsto (0,0,1,0)$ and hadron $\mapsto (0,0,0,1)$ (function OneHot of Algorithm 2). Every two different categories are now at the same distance, $\|\mathbf{e}_k - \mathbf{e}_l\| = \sqrt{2}$, and their inner product is $\mathbf{e}_k^{\mathsf{T}}\mathbf{e}_l = 0$; the code carries no information except identity. An example with several attributes concatenates the codes: a particle type (4 categories) and a sub-system (3 categories) give a vector of length 7 with exactly two ones. The same encoding

represents the *targets* of an m-class problem as rows of an $n \times m$ matrix Y with $Y_{ik} = \mathbf{1}\{y_i = k\}$; averaging rows gives class frequencies (Notes 5, 6 and 9).

3.2 Texts: the bag of words

The *bag of words* represents a document by how often each word of a fixed vocabulary occurs in it. It is a sum of one-hot vectors, one per word occurrence, so it inherits the properties above and adds one decision: word order is thrown away.

Algorithm 2 One-hot codes and bag-of-words vectors from an index fitted on training data

Require: training items: category values $v_1, \dots, v_n$, or documents $T_1, \dots, T_n$ with a tokeniser

function FitIndex(training items)
  $W \leftarrow$ set of all values (or of all tokens of all documents) ▷ average $O(1)$ per insertion
  sort W; index$[w] \leftarrow$ position of w in W; $V \leftarrow |W|$ ▷ training data only
  return (index, V)
end function
function OneHot((index, V), v)
  $\mathbf{e} \leftarrow (0, \dots, 0) \in \mathbb{R}^V$; if v is a key of index, $e_{\text{index}[v]} \leftarrow 1$
  return e ▷ an unseen value stays all-zero
end function
function BagOfWords((index, V), T)
  $\mathbf{x} \leftarrow$ empty sparse vector of length V ▷ stores only non-zero entries
  for each token t of T that is a key of index **do**
    $x_{\text{index}[t]} \leftarrow x_{\text{index}[t]} + 1$
  end for
  return x ▷ the sum of the one-hot codes of the tokens
end function

Worked example — three sentences become three vectors

Training documents: T_1 = "the muon hit the detector", T_2 = "a photon hit the detector", T_3 = "the muon decayed".

1. *Tokenise*: lower-case and split at spaces. The three documents have 5, 5 and 3 tokens.
2. *Build the vocabulary* from the training documents, here sorted alphabetically: (a, decayed, detector, hit, muon, photon, the), so $V = 7$.
3. *Count*: entry j of $\mathbf{x}_i$ is the number of occurrences of word j in T_i:

$$\mathbf{x}_1 = (0, 0, 1, 1, 1, 0, 2), \qquad \mathbf{x}_2 = (1, 0, 1, 1, 0, 1, 1), \qquad \mathbf{x}_3 = (0, 1, 0, 0, 1, 0, 1).$$

   The entries of $\mathbf{x}_i$ add up to the length of T_i.
4. *A new document* "the pion hit the detector" becomes $(0, 0, 1, 1, 0, 0, 2)$: "pion" is not in the vocabulary and is dropped (or counted in an extra "unknown" entry).

The vectors are compared in Section 6. Two lessons are visible already. First, "dog bites man" and "man bites dog" get identical vectors; the representation cannot tell them apart. A representation that keeps one vector per position (the sequence of one-hot or embedding vectors, padded to a fixed length and flattened), or one that also counts adjacent word pairs (*bigrams*: "dog bites", "bites man"), does. Second, frequent short words ("the", "a") carry little meaning but

dominate the counts; they are often removed as *stop words* or down-weighted, for instance by the tf–idf scheme, which multiplies each count by $\log(n/n_j)$, where n_j is the number of training documents containing word j, so that a word present in every document gets weight zero.

Running time — one-hot codes and bag of words (Algorithm 2)

Let n_{it} be the number of items seen by FitIndex ($n_{\text{it}} = n$ for a categorical column, the total number of tokens for documents) and ℓ the length of one document. **Fitting the index** $O(n_{\text{it}} + V \log V)$: one average-$O(1)$ hash-set insertion per item, then a sort of the V distinct values. **Encoding**: $O(1)$ for one categorical value and $O(\ell)$ for a document if the output is sparse; writing a dense vector adds $O(V)$. **Memory**: dense codes take $O(nV)$ (for a categorical column $V = K$, the number of categories); sparse ones take $O(n)$ for a categorical column and $O(n_{\text{it}})$ for documents, one entry per distinct token of each document. Example: $n = 10^5$ documents, $V = 5 \times 10^4$ words and about 200 distinct words per document need 20 GB as a dense `float32` matrix but only $10^5 \times 200 \times 8$ bytes $= 160$ MB sparse (a 4-byte index and a 4-byte value per non-zero entry).

3.3 Events with a variable number of objects

A collision event is a list of objects whose length varies. A fixed d is obtained by keeping the k leading objects (by transverse momentum p_{T}), padded with zeros (Algorithm 3), or by computing summary observables.

Algorithm 3 A fixed-length vector from an event with a variable number of objects

Require: event: r objects, each described by f numbers with p_{T} first; number k of objects kept

1: **function** LeadingObjects(event, k)
2: sort the objects by decreasing p_{T} ▷ or select the k largest without a full sort
3: keep the first $\min(r, k)$ objects; append $k - \min(r, k)$ objects of f zeros ▷ zero padding
4: **return** the k objects concatenated into one vector of length kf
5: **end function**

Running time — leading objects (Algorithm 3)

Training: none, nothing is fitted (k is a choice). **Encoding** $O(r \log r + kf)$ per event: the sort, then writing kf numbers; partial selection lowers the sort to $O(r + k \log k)$. **Memory** $O(kf)$ per event, $O(nkf)$ for the dataset.

Worked example — an event with three jets

Three jets have $(p_{\text{T}}, \eta, \phi) = (120\ \text{GeV}, 0.5, 0.0)$, $(80\ \text{GeV}, -0.3, 3.0)$ and $(30\ \text{GeV}, 1.8, 1.2)$, with $\eta = -\ln\tan(\theta/2)$ the pseudorapidity, θ the polar angle to the beam (unrelated to the $\eta(\mathbf{x})$ of Section 4.1), and ϕ the azimuth in radians. A summary vector with $d = 5$ holds the multiplicity, $H_{\text{T}} = \sum p_{\text{T}} = 230$ GeV, the two leading p_{T} and the invariant mass of the two leading jets, which for massless jets is

$$m_{jj}^2 = 2p_{\text{T}1}p_{\text{T}2}\big(\cosh(\eta_1 - \eta_2) - \cos(\phi_1 - \phi_2)\big) = 2 \times 120 \times 80 \times \big(\cosh 0.8 - \cos(-3.0)\big)$$
$$= 19\,200 \times \big(1.3374 - (-0.9900)\big) = 19\,200 \times 2.3274 \approx 44\,687\ \text{GeV}^2,$$

so $m_{jj} \approx 211.4$ GeV and $\mathbf{x} = (3, 230, 120, 80, 211.4)$. "Two leading jets, padded" gives instead $\mathbf{x} = (120, 0.5, 0.0, 80, -0.3, 3.0)$. Both forget the third jet partly or completely — the price of a fixed d. An event with a single jet, $(120\ \text{GeV}, 0.5, 0.0)$, becomes $(120, 0.5, 0.0, 0, 0, 0)$: the padding looks like a real jet with $p_{\text{T}} = 0$ at $\eta = \phi = 0$, and distances treat it as one.

Learned representations. Deep networks learn the features themselves from raw inputs. Their intermediate vectors — *embeddings* — are reused far beyond the task they were trained for: a token from a vocabulary of 10^4–10^5 pieces becomes a vector of a few hundred to over ten thousand dimensions (768 in BERT-base [5]), and CLIP maps images and captions into a *shared* space where matching pairs are close [13]. Looking a token up in an embedding table is a matrix product in disguise: if $\mathbf{e}_k \in \mathbb{R}^V$ is the one-hot code of token k and W is the $V \times d_{\text{emb}}$ embedding matrix, then $\mathbf{e}_k^{\mathsf{T}} W$ is row k of W. The product is a table lookup, so neither $\mathbf{e}_k$ nor the multiplication is ever computed.

Levels of representation. The representations of this section can be ordered by level. Lowest are the raw measurements: pixels, calorimeter cells, the constituents of a jet. In the middle are observables computed from them by hand: the dark-pixel count and box size of the 3×3 images, H_{T} and m_{jj} of the three-jet event. Highest are learned embeddings. A higher level keeps what matters for the task and discards what does not (such as the position of a bar), so simple methods built on distances or linear rules work better on it: 1-NN failed on the raw pixels and succeeded on the features. Raw high-dimensional inputs also suffer from the curse of dimensionality (Note 3, Section 3); neural networks (week 5) learn the levels layer by layer.

> **Modern connection — representations as a product**
>
> A common recipe in 2026 is to keep a large pre-trained model frozen and train only a simple classifier — or a nearest-neighbour rule (Note 5) — on its embeddings: the algorithms of weeks 1–3 are what is applied on top.

4 Families of learning problems

Supervised learning. The goal is to predict a target y from an input $\mathbf{x}$. In *classification* y is a category (binary, $y \in \{0, 1\}$ or $\{-1, +1\}$; or multiclass, $y \in \{1, \dots, m\}$). In *regression* y is a real number or a vector of real numbers.

Unsupervised learning. There is no explicit target. *Density estimation* models the distribution $p(\mathbf{x})$ of the inputs; *clustering* groups similar examples; *dimensionality reduction* finds a few coordinates that describe the data (for visualisation or compression); *representation learning* builds useful features.

Between the two. *Semi-supervised* learning uses a few labelled and many unlabelled examples. *Self-supervised* learning manufactures targets from the data: the next token of a text, a masked image patch. Next-token prediction is multiclass classification with m the vocabulary size. The model outputs a probability $f_k(\mathbf{x})$ for every token k and is trained to minimise the *log loss* (cross-entropy) $-\log f_y(\mathbf{x})$, minus the logarithm of the probability it gave to the token y that actually came next (Table 2; Note 4, Section 2). This is the negative log-likelihood of the density-estimation row, applied to the conditional distribution $P(Y \mid \mathbf{X} = \mathbf{x})$; the simple losses below thus also drive the largest models.

Reinforcement learning. An agent chooses actions to maximise a cumulative reward, as when language models are fine-tuned from human preferences [12]; it is outside the scope of this course.

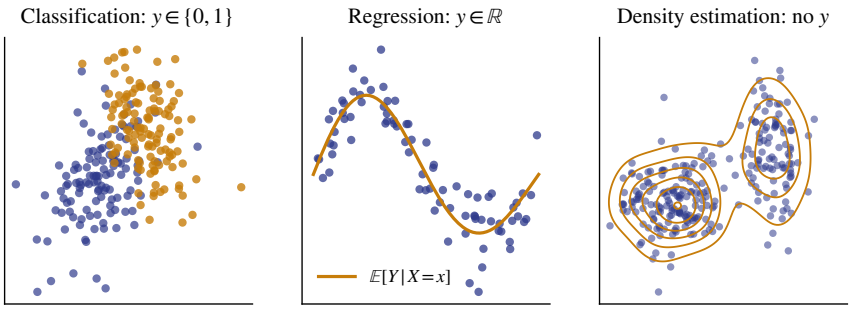


Figure 1. Three tasks. Left: classify events into signal (amber) and background (indigo) from two observables. Middle: regression; the input is a single number x and the curve is the conditional mean $\mathbb{E}[Y \mid X = x]$ that a good regressor approximates. Right: density estimation; contours of an estimate of $p(\mathbf{x})$ (Note 5 explains how it was computed).

Table 2. The three classic problems. This table is the reference for the rest of the course.

	Target	What $f(\mathbf{x})$ should be	Typical loss L
Classification	$y \in \{1, \dots, m\}$	the most probable class, or the vector of probabilities $P(Y = k \mid \mathbf{X} = \mathbf{x})$	error $\mathbf{1}\{f(\mathbf{x}) \neq y\}$; for probabilities, log loss $-\log f_y(\mathbf{x})$
Regression	$y \in \mathbb{R}$	the conditional mean $\mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$	squared error $(f(\mathbf{x}) - y)^2$
Density estimation	none	the density $p(\mathbf{x})$	negative log-likelihood $-\log f(\mathbf{x})$

What kind of function is learned. A classifier maps $\mathbb{R}^d \to \{1, \dots, m\}$ (or to probability vectors), a regressor $\mathbb{R}^d \to \mathbb{R}$, a density estimator $\mathbb{R}^d \to [0, \infty)$ with $\int f(\mathbf{x})\, d\mathbf{x} = 1$; the family searched is chosen as in Note 4, Section 1.

4.1 Why these targets: two derivations

The column "What $f(\mathbf{x})$ should be" of Table 2 is the prediction with the smallest *expected* value of the loss in the column "Typical loss" when $P(\mathbf{X}, Y)$ is known. Both derivations work at a fixed input $\mathbf{x}$, where $f(\mathbf{x})$ is just a number c to choose.

Squared error. Write $\mu(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ and abbreviate $\mu = \mu(\mathbf{x})$. Add and subtract μ inside the square and expand:

$$(c - Y)^2 = \big((c - \mu) + (\mu - Y)\big)^2 = (c - \mu)^2 + 2(c - \mu)(\mu - Y) + (\mu - Y)^2.$$

Now take the conditional expectation given $\mathbf{X} = \mathbf{x}$ term by term; expectation is linear and $c - \mu$ is a constant, so

$$\mathbb{E}\big[(c - Y)^2 \mid \mathbf{X} = \mathbf{x}\big] = (c - \mu)^2 + 2(c - \mu)\,\mathbb{E}[\mu - Y \mid \mathbf{X} = \mathbf{x}] + \mathbb{E}\big[(\mu - Y)^2 \mid \mathbf{X} = \mathbf{x}\big].$$

The middle expectation is $\mu - \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}] = \mu - \mu = 0$, so the cross term vanishes, and the last one is, by definition, the conditional variance. Hence

$$\mathbb{E}\big[(c - Y)^2 \mid \mathbf{X} = \mathbf{x}\big] = \big(c - \mu(\mathbf{x})\big)^2 + \mathrm{Var}(Y \mid \mathbf{X} = \mathbf{x}). \tag{1}$$

The first term is zero exactly when $c = \mu(\mathbf{x})$, and the second does not depend on c at all. The same answer follows from one derivative: $g(c) = \mathbb{E}[(c - Y)^2 \mid \mathbf{X} = \mathbf{x}] = c^2 - 2c\,\mu + \mathbb{E}[Y^2 \mid \mathbf{X} = \mathbf{x}]$ has

$g'(c) = 2(c-\mu) = 0$ at $c = \mu$, and $g''(c) = 2 > 0$ makes it a minimum. So the best regressor is the conditional mean, and $\mathrm{Var}(Y \mid \mathbf{X} = \mathbf{x})$ is noise that *no* predictor can remove. For any predictor f, set $c = f(\mathbf{x})$ in (1), which holds at every $\mathbf{x}$, and average over $\mathbf{X}$ with the law of total expectation $\mathbb{E}\big[\mathbb{E}[Z \mid \mathbf{X}]\big] = \mathbb{E}[Z]$:

$$\mathbb{E}\big[(f(\mathbf{X}) - Y)^2\big] = \mathbb{E}\big[(f(\mathbf{X}) - \mu(\mathbf{X}))^2\big] + \mathbb{E}\big[\,\mathrm{Var}(Y \mid \mathbf{X})\big], \tag{2}$$

the sum of a part we can reduce by learning and a floor set by nature. Note 4, Section 6, splits the first part further into bias and variance.

Worked example — a regression problem small enough to solve exactly

The input takes two values, $\mathbf{X} = 1$ or $\mathbf{X} = 2$, each with probability $\frac{1}{2}$. Given $\mathbf{X} = 1$, Y is 1 or 3 with equal probability; given $\mathbf{X} = 2$, Y is 2, 4 or 6 with equal probability.

- Conditional means: $\mu(1) = (1+3)/2 = 2$ and $\mu(2) = (2+4+6)/3 = 4$.
- Conditional variances: $\mathrm{Var}(Y \mid \mathbf{X} = 1) = \frac{1}{2}\big[(1-2)^2 + (3-2)^2\big] = 1$ and $\mathrm{Var}(Y \mid \mathbf{X} = 2) = \frac{1}{3}\big[4 + 0 + 4\big] = \frac{8}{3}$.
- The best possible mean squared error is $\mathbb{E}[\mathrm{Var}(Y \mid \mathbf{X})] = \frac{1}{2}\big(1 + \frac{8}{3}\big) = \frac{11}{6} \approx 1.83$, reached by $f = \mu$.
- The best constant predictor, $f \equiv \mathbb{E}[Y] = \frac{1}{2}\cdot 2 + \frac{1}{2}\cdot 4 = 3$ (the average of the conditional means), ignores $\mathbf{X}$. Directly: at $\mathbf{X} = 1$ its squared errors are 4 and 0 (mean 2); at $\mathbf{X} = 2$ they are $1, 1, 9$ (mean $\frac{11}{3}$); overall $\frac{1}{2}(2 + \frac{11}{3}) = \frac{17}{6} \approx 2.83$. With (2): $\frac{11}{6} + \frac{1}{2}\big[(3-2)^2 + (3-4)^2\big] = \frac{11}{6} + 1 = \frac{17}{6}$, the same number.

Using the input reduces the error from $\frac{17}{6}$ to $\frac{11}{6}$; nothing reduces it further.

Classification error. Write $\eta_k(\mathbf{x}) = P(Y = k \mid \mathbf{X} = \mathbf{x})$ for the class probabilities at $\mathbf{x}$. If we predict class c, the loss $\mathbf{1}\{c \neq Y\}$ is 1 for every outcome $Y \neq c$ and 0 for $Y = c$, so

$$\mathbb{E}\big[\mathbf{1}\{c \neq Y\} \mid \mathbf{X} = \mathbf{x}\big] = \sum_{k=1}^{m} \eta_k(\mathbf{x})\,\mathbf{1}\{c \neq k\} = \sum_{k \neq c} \eta_k(\mathbf{x}) = 1 - \eta_c(\mathbf{x}),$$

using $\sum_k \eta_k(\mathbf{x}) = 1$ in the last step. The conditional error is smallest when $\eta_c(\mathbf{x})$ is largest, so the best classifier — the *Bayes classifier* — is

$$f^*(\mathbf{x}) = \arg\max_k \eta_k(\mathbf{x}), \qquad \text{with conditional error } 1 - \max_k \eta_k(\mathbf{x}). \tag{3}$$

Averaging over $\mathbf{X}$ gives the *Bayes error* $R^* = \mathbb{E}\big[1 - \max_k \eta_k(\mathbf{X})\big]$, the smallest error rate any classifier can achieve with these inputs. With two classes and $\eta(\mathbf{x})$ the probability of one of them it reads $R^* = \mathbb{E}[\min(\eta(\mathbf{X}), 1 - \eta(\mathbf{X}))] \leq \frac{1}{2}$. How to estimate the η_k from data is the subject of Note 6.

Worked example — Bayes classifier and Bayes error from a table

A muon chamber has two detection layers; $X \in \{0, 1, 2\}$ is the number of layers that fired, and $Y = 1$ if the particle was a muon. The joint distribution is known (from simulation); in the table $P(x, Y{=}0)$ abbreviates $P(X{=}x, Y{=}0)$, and $P(x)$ is $P(X{=}x)$:

x	$P(x, Y{=}0)$	$P(x, Y{=}1)$	$P(x)$	$\eta(x) = P(Y{=}1 \mid x)$	$f^*(x)$	error at x
0	0.45	0.05	0.50	0.10	0	0.10
1	0.12	0.18	0.30	0.60	1	0.40
2	0.04	0.16	0.20	0.80	1	0.20

Each $\eta(x)$ is a row's $Y{=}1$ entry divided by its total, e.g. $0.18/0.30 = 0.60$. The Bayes classifier predicts "muon" when $\eta(x) > \frac{1}{2}$, i.e. for $x \geq 1$. Its error is

$$R^* = 0.50 \times 0.10 + 0.30 \times 0.40 + 0.20 \times 0.20 = 0.05 + 0.12 + 0.04 = 0.21,$$

which is also the sum of the *smaller* joint entry in each row. Two comparisons: the stricter rule "muon only if $x = 2$" has error $0.05 + 0.18 + 0.04 = 0.27$; ignoring X and always predicting the more frequent class (no muon, since $P(Y{=}1) = 0.39$) has error 0.39. No classifier using only X beats 21%; a third layer — a better representation — could.

Density estimation. A good density is large where data actually fall: $-\log f(\mathbf{x})$ is large when the observed $\mathbf{x}$ was given a low density, so its average penalises a density that misses the data. The expected negative log-likelihood $\mathbb{E}[-\log f(\mathbf{X})]$ is minimised by the true density, $f = p$ (Gibbs' inequality: $\mathbb{E}[-\log f(\mathbf{X})] \geq \mathbb{E}[-\log p(\mathbf{X})]$ for every density f, with equality only for $f = p$); maximum likelihood (Note 2, Section 2) minimises its empirical version.

The Bayes error belongs to the representation The irreducible errors $\mathbb{E}[\mathrm{Var}(Y \mid \mathbf{X})]$ and R^* belong to the joint distribution of the chosen inputs and the target, not to any algorithm: a better algorithm can approach them but never beat them; a better *representation* — an extra, informative feature — can lower them.

5 Anatomy of a machine-learning project

1. **Formulate.** Which task of Table 2 is it? Which loss reflects what you actually care about (a missed signal event and a false alarm rarely cost the same)?
2. **Understand the data.** What is one example, how were the labels obtained, and what is the unit of independence?
3. **Split first.** Set aside validation and test data *before* looking for patterns (Note 4).
4. **Pre-process using training data only.** Scaling constants, vocabularies and feature selections are part of the model; computing them on all the data leaks information from the test set.
5. **Start simple.** A constant predictor, a linear model or a nearest-neighbour rule gives a baseline; test new code first on a two-input toy problem that you can see in a scatter plot.
6. **Evaluate out of sample**, compare models fairly and report uncertainty (Note 4).
7. **Deploy and monitor.** The data distribution can drift after deployment.

5.1 Leakage, with numbers

Leakage is any route by which information about the evaluation data reaches the model during its construction. It makes the *reported* performance better than the performance on genuinely new data.

Worked example — the same galaxy on both sides of the split

A morphology dataset contains 1000 images: 250 galaxies, each photographed 4 times. It is shuffled and split at random into 800 training and 200 test images. Pick a test image. Given that it is in the test set, the other 199 test images are a random subset of the remaining 999, every subset equally likely. Its 3 siblings are all in that subset for $\binom{199}{3}$ of the $\binom{999}{3}$ equally likely sets of positions they can occupy, so the probability that *all three* siblings also landed in the test set is (the 3! in both binomials cancels)

$$\frac{\binom{199}{3}}{\binom{999}{3}} = \frac{199 \cdot 198 \cdot 197}{999 \cdot 998 \cdot 997} \approx 0.0078.$$

Thus 99.2% of test images have at least one image of the same galaxy in the training set. Suppose a classifier is right 98% of the time on galaxies it has seen and 70% of the time on new galaxies. The random split reports $0.992 \times 0.98 + 0.008 \times 0.70 \approx 97.8\%$, whereas a survey of new galaxies will see 70%. The fix is a *grouped* split that keeps all images of a galaxy on the same side (Note 4, Section 4); it then reports 70%, the honest number.

Validation and cross-validation. Data used to *choose* between models or settings are *validation* data; test data are touched only once, at the end, to report the final number. When data are scarce, *k-fold cross-validation* cuts them into k equal parts (*folds*); each fold in turn serves as validation data for a model trained on the other $k - 1$, and the k validation errors are averaged (Note 4, Section 4). Everything fitted, including a feature selection, must then be fitted anew inside each fold.

Worked example — choosing features before cross-validation

Take $n = 50$ examples with balanced random labels and 5000 features of pure noise, so that no classifier can do better than chance (50% error) on new data. A common mistake is to first keep the 20 features most correlated with the label, using all 50 examples, and then estimate the error of a 1-nearest-neighbour classifier (Note 5) by 5-fold cross-validation on those features. Averaged over 200 simulated datasets (our own simulation), this procedure reports an error of about 9%. Repeating the feature selection *inside* each fold, on the training part only, reports about 52%, close to the true 50%. The selection step had seen the labels of the validation examples and picked features that happen to fit them. This is a smaller version of the example in [7, section 7.10.2], where keeping 100 features drives the reported error down to about 3%.

Other common leaks are features that encode the label by accident (a file name, a simulation run number) and random splits of time series. The remedy is always Algorithm 4: everything *fitted* — scaling constants, vocabularies, selected features, the model — is fitted on training data and *applied* unchanged to the evaluation data.

Running time — leak-free evaluation (Algorithm 4)

Write $T_{\text{FIT}}, T_{\text{APPLY}}, T_{\text{TRAIN}}, T_{\text{PREDICT}}$ for the costs of the four procedures and n_{tr} for the training size. **Training** $T_{\text{FIT}}(n_{\text{tr}}) + n_{\text{tr}}\, T_{\text{APPLY}} + T_{\text{TRAIN}}(n_{\text{tr}})$: fit ϕ, transform every training input once, train. **Prediction** $T_{\text{APPLY}} + T_{\text{PREDICT}}$ per test example: the same transformation, then the model. **Memory**: that of ϕ (e.g. $O(d)$ for standardisation constants, $O(V)$ for a vocabulary) plus that of θ. The leaky version costs the same, so doing it right is free; k-fold cross-validation repeats the training part k times.

Algorithm 4 Leak-free hold-out evaluation of a pipeline

Require: dataset D_n; pre-processing Fit/Apply; learner Train/Predict; loss L

1: split D_n into D_{train} and D_{test} ▷ by group or by time when the data are not i.i.d.
2: $\phi \leftarrow \textsc{Fit}(D_{\text{train}})$ ▷ e.g. means and standard deviations, vocabulary, selected features
3: $\theta \leftarrow \textsc{Train}\big(\{(\textsc{Apply}(\phi, \mathbf{x}), y) : (\mathbf{x}, y) \in D_{\text{train}}\}\big)$
4: **return** $\frac{1}{|D_{\text{test}}|} \sum_{(\mathbf{x},y)\in D_{\text{test}}} L\big(\textsc{Predict}(\theta, \textsc{Apply}(\phi, \mathbf{x})), y\big)$

k-fold version: cut D_n into k folds; for each fold run lines 2–4 with that fold as the evaluation set and the rest as D_{train} (Fit includes any feature selection), then average the k results (Note 4, Section 4).

Modern connection — leakage in published science

A survey of machine-learning-based science found leakage — pre-processing on the full dataset, duplicates across splits, features that encode the label — in hundreds of papers in seventeen fields, inflating reported performance and, in some case studies, making a model no better than simple baselines look excellent [8]. Steps 3 and 4 above and documenting how data were collected [6] are the cheapest insurance.

6 A vector toolkit

A vector has three faces in this course. Mathematically it is an element of $\mathbb{R}^d$. In code it is a `numpy` array of shape `(d,)`, taking $4d$ bytes in `float32` or $8d$ in `float64` — for instance one row of the design matrix. Graphically it is a point of a scatter plot (Section 2) or an arrow from the origin to that point, the picture behind the angles and projections below.

For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ the *inner product* is $\mathbf{a}^{\mathsf{T}}\mathbf{b} = \sum_{j=1}^{d} a_j b_j$. The L_p *norm* is $\|\mathbf{a}\|_p = \big(\sum_j |a_j|^p\big)^{1/p}$ for $p \geq 1$; $p = 2$ gives the Euclidean norm $\|\mathbf{a}\| = \sqrt{\mathbf{a}^{\mathsf{T}}\mathbf{a}}$, $p = 1$ the sum of absolute values and $p \to \infty$ the largest absolute value. The Euclidean distance is $\|\mathbf{a} - \mathbf{b}\|$.

The *cosine similarity* $\cos(\mathbf{a}, \mathbf{b}) = \mathbf{a}^{\mathsf{T}}\mathbf{b}/(\|\mathbf{a}\|\,\|\mathbf{b}\|)$ compares directions only — the standard way to compare embeddings. Geometrically, $\mathbf{a}^{\mathsf{T}}\mathbf{b} = \|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta$ with θ the angle between the vectors, and $\mathbf{a}^{\mathsf{T}}\mathbf{b}/\|\mathbf{b}\|$ is the signed length of the projection of $\mathbf{a}$ onto the direction of $\mathbf{b}$. The Cauchy–Schwarz inequality $|\mathbf{a}^{\mathsf{T}}\mathbf{b}| \leq \|\mathbf{a}\|\,\|\mathbf{b}\|$ guarantees $-1 \leq \cos(\mathbf{a}, \mathbf{b}) \leq 1$; the value is 1 for parallel vectors, 0 for orthogonal ones and -1 for opposite ones. Note 5, Section 5, compares the norms' unit balls.

Worked example — norms, inner product and angle

Let $\mathbf{a} = (3, -4, 0, 12)$ and $\mathbf{b} = (1, 2, 2, 0)$ in $\mathbb{R}^4$.

- $\|\mathbf{a}\|_1 = 3 + 4 + 0 + 12 = 19$, $\|\mathbf{a}\|_2 = \sqrt{9 + 16 + 0 + 144} = \sqrt{169} = 13$, $\|\mathbf{a}\|_\infty = 12$. Always $\|\mathbf{a}\|_\infty \leq \|\mathbf{a}\|_2 \leq \|\mathbf{a}\|_1$.
- $\|\mathbf{b}\| = \sqrt{1 + 4 + 4} = 3$ and $\mathbf{a}^{\mathsf{T}}\mathbf{b} = 3 - 8 + 0 + 0 = -5$.
- $\cos(\mathbf{a}, \mathbf{b}) = -5/(13 \times 3) = -5/39 \approx -0.128$, an angle of about $97.4°$: the vectors are nearly orthogonal. The projection of $\mathbf{a}$ on the direction of $\mathbf{b}$ has length $-5/3$.
- $\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}^{\mathsf{T}}\mathbf{b} = 169 + 9 + 10 = 188$; directly, $\mathbf{a} - \mathbf{b} = (2, -6, -2, 12)$ and $4 + 36 + 4 + 144 = 188$.

Distance or direction. The two notions can disagree (Figure 2, right). For $\mathbf{u} = (1, 2)$, the vector $\mathbf{w} = (2, 1)$ is close in distance ($\|\mathbf{u} - \mathbf{w}\| = \sqrt{2} \approx 1.41$) but points elsewhere ($\cos = 0.8$), while $3\mathbf{u}$ is far ($\|\mathbf{u} - 3\mathbf{u}\| = 2\sqrt{5} \approx 4.47$) but points the same way ($\cos = 1$). For bag-of-words vectors, a document copied three times is $3\mathbf{x}$: its topic is unchanged, and only the cosine sees that. On the three sentences of Section 3.2, $\mathbf{x}_1^\mathsf{T}\mathbf{x}_2 = 4$, $\mathbf{x}_1^\mathsf{T}\mathbf{x}_3 = 3$, $\mathbf{x}_2^\mathsf{T}\mathbf{x}_3 = 1$ and $\|\mathbf{x}_1\|^2 = 7$, $\|\mathbf{x}_2\|^2 = 5$, $\|\mathbf{x}_3\|^2 = 3$, so

$$\cos(\mathbf{x}_1, \mathbf{x}_2) = \frac{4}{\sqrt{35}} \approx 0.68, \qquad \cos(\mathbf{x}_1, \mathbf{x}_3) = \frac{3}{\sqrt{21}} \approx 0.65, \qquad \cos(\mathbf{x}_2, \mathbf{x}_3) = \frac{1}{\sqrt{15}} \approx 0.26.$$

Removing the stop words "the" and "a" leaves the vocabulary (decayed, detector, hit, muon, photon) and the vectors $(0, 1, 1, 1, 0)$, $(0, 1, 1, 0, 1)$ and $(1, 0, 0, 1, 0)$; the cosines become $2/3 \approx 0.67$, $1/\sqrt{6} \approx 0.41$ and 0: T_1 is now clearly closer to T_2 (shared "hit", "detector") than to T_3 (shared "muon"), and T_2, T_3 share no content word.

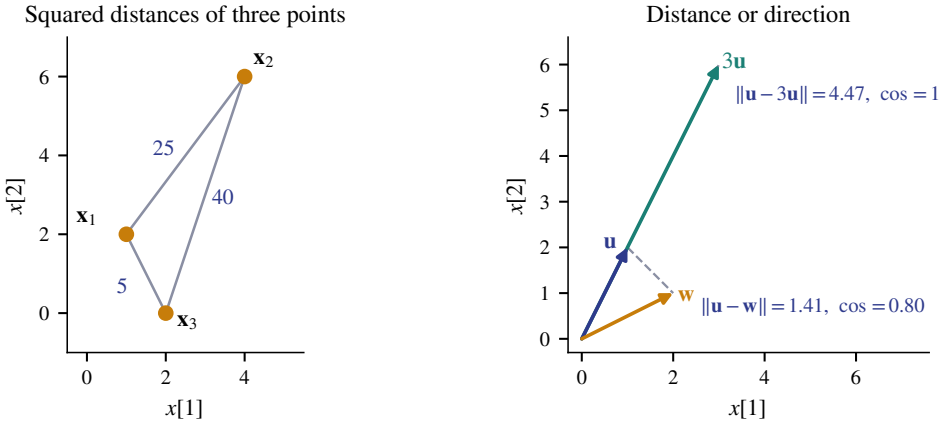


Figure 2. Left: the three points of the worked example below, with their squared distances. Right: Euclidean distance and cosine similarity disagree; $\mathbf{w}$ is nearer to $\mathbf{u}$, but $3\mathbf{u}$ has the same direction.

6.1 Distances in bulk

Many methods need the distances between q query points (rows of $Q \in \mathbb{R}^{q \times d}$) and n data points (rows of X). For one pair, expand the square of the difference using the bilinearity of the inner product:

$$\|\mathbf{a} - \mathbf{b}\|^2 = (\mathbf{a} - \mathbf{b})^\mathsf{T}(\mathbf{a} - \mathbf{b}) = \mathbf{a}^\mathsf{T}\mathbf{a} - \mathbf{a}^\mathsf{T}\mathbf{b} - \mathbf{b}^\mathsf{T}\mathbf{a} + \mathbf{b}^\mathsf{T}\mathbf{b} = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}^\mathsf{T}\mathbf{b},$$

where the middle step uses $\mathbf{b}^\mathsf{T}\mathbf{a} = \mathbf{a}^\mathsf{T}\mathbf{b}$. Apply this to every pair $(\mathbf{q}_i, \mathbf{x}_j)$. With the vectors of squared norms $\mathbf{s}^Q \in \mathbb{R}^q$ and $\mathbf{s}^X \in \mathbb{R}^n$, with components $s_i^Q = \|\mathbf{q}_i\|^2$ and $s_j^X = \|\mathbf{x}_j\|^2$, and since $(QX^\mathsf{T})_{ij} = \mathbf{q}_i^\mathsf{T}\mathbf{x}_j$, the whole $q \times n$ matrix of squared distances is

$$D = \mathbf{s}^Q \mathbf{1}_n^\mathsf{T} + \mathbf{1}_q (\mathbf{s}^X)^\mathsf{T} - 2\,QX^\mathsf{T}. \tag{4}$$

Here $\mathbf{1}_n$ is the vector of n ones. The outer product $\mathbf{s}^Q\mathbf{1}_n^\mathsf{T}$ is the $q \times n$ matrix whose entry (i, j) is s_i^Q (every column equals $\mathbf{s}^Q$), and $\mathbf{1}_q(\mathbf{s}^X)^\mathsf{T}$ has entry (i, j) equal to s_j^X (every row equals $(\mathbf{s}^X)^\mathsf{T}$). Entry (i, j) of D is therefore $s_i^Q + s_j^X - 2\,\mathbf{q}_i^\mathsf{T}\mathbf{x}_j$, the single-pair identity; in code the outer products are never formed, `numpy` "broadcasts" the two norm vectors instead. Below, $\odot$ is the element-wise (Hadamard) product. When $Q = X$, the matrix $G = XX^\mathsf{T}$ of all inner products is called the *Gram matrix*; it reappears with the kernels of Note 10. Algorithm 5 sets this computation against the naive triple loop.

Algorithm 5 Pairwise squared distances: the naive triple loop and the vectorised form

Require: queries $Q \in \mathbb{R}^{q\times d}$, data $X \in \mathbb{R}^{n\times d}$
Ensure: $D \in \mathbb{R}^{q\times n}$ with $D_{ij} = \|\mathbf{q}_i - \mathbf{x}_j\|^2$

function SqDistLoop(Q, X)
  for $i = 1, \dots, q$ and $j = 1, \dots, n$ **do**
    $D_{ij} \leftarrow 0$
    for $k = 1, \dots, d$ **do**
      $D_{ij} \leftarrow D_{ij} + (Q_{ik} - X_{jk})^2$
    end for
  end for
  return D
end function
function SqDist(Q, X)
  $\mathbf{s}^Q \leftarrow$ row sums of $Q \odot Q$; $\mathbf{s}^X \leftarrow$ row sums of $X \odot X$ ▷ $O((q+n)d)$
  $P \leftarrow QX^\mathsf{T}$ ▷ one matrix product, $O(qnd)$
  $D_{ij} \leftarrow \max(s_i^Q + s_j^X - 2P_{ij},\ 0)$ for all i, j ▷ broadcasting, $O(qn)$; clip round-off
  return D
end function

Large q: call SqDist on blocks of b query rows and keep only what is needed (e.g. the k smallest entries per row); memory $O(bn)$ instead of $O(qn)$.
Cosine similarities: divide every row of Q and of X by its norm; the matrix is then QX^T.

Running time — pairwise squared distances (Algorithm 5)

Operations: both need $O(qnd)$ — the loop does three operations (subtract, square, add) per index triple; the vectorised version about $2qnd$ in QX^T plus lower-order work. **Memory**: inputs $O((q+n)d)$, output $O(qn)$; computing the differences by broadcasting, `(Q[:,None,:] - X[None,:,:])**2`, would materialise a $q \times n \times d$ array. **In practice**: on the machine used to prepare these notes, with $q = 200$, $n = 2000$ and $d = 64$, a pure-Python triple loop took a few seconds, the matrix-product version a few milliseconds — a factor of several hundred to a few thousand — and the broadcasting version, which allocates 205 MB, was roughly ten times slower than the matrix product. **Scale**: with $Q = X$ and $n = 10^5$ points, D has 10^{10} entries, 40 GB in single precision; this is why the blocked form, and for very large n approximate neighbour search (Note 5, Section 2), are needed.

Worked example — a squared-distance matrix by hand and by the identity

Three points in $\mathbb{R}^2$: $\mathbf{x}_1 = (1, 2)$, $\mathbf{x}_2 = (4, 6)$, $\mathbf{x}_3 = (2, 0)$ (Figure 2, left). *Directly*: $\mathbf{x}_1 - \mathbf{x}_2 = (-3, -4)$ gives 25; $\mathbf{x}_1 - \mathbf{x}_3 = (-1, 2)$ gives 5; $\mathbf{x}_2 - \mathbf{x}_3 = (2, 6)$ gives 40. *By the identity*, with $Q = X$ and hence $\mathbf{s}^Q = \mathbf{s}^X = \mathbf{s}$: the squared norms are $\mathbf{s} = (5, 52, 4)$ and the Gram matrix is

$$G = XX^\mathsf{T} = \begin{pmatrix} 5 & 16 & 2 \\ 16 & 52 & 8 \\ 2 & 8 & 4 \end{pmatrix}, \qquad \begin{aligned} D_{12} &= 5 + 52 - 2\cdot 16 = 25, \\ D_{13} &= 5 + 4 - 2\cdot 2 = 5, \\ D_{23} &= 52 + 4 - 2\cdot 8 = 40, \end{aligned} \qquad D = \begin{pmatrix} 0 & 25 & 5 \\ 25 & 0 & 40 \\ 5 & 40 & 0 \end{pmatrix}.$$

The diagonal of G is $\mathbf{s}$; the diagonal of D is zero ($D_{ii} = s_i + s_i - 2G_{ii}$) and D is symmetric, as it must be. The nearest neighbour of $\mathbf{x}_1$ is $\mathbf{x}_3$ — read off as the smallest off-diagonal entry of row 1.

A numerical caveat. Identity (4) subtracts large numbers to obtain a small one. `float32` keeps 24 significant bits (about 7 decimal digits): representable numbers between 2^{19} and 2^{20} (around 10^6) are $2^{19-23} = 0.0625$ apart, between 2^{20} and 2^{21} 0.125 apart. Take $a = 1000.1$ and $b = 1000.0$. Already a is stored as 1000.09998; with this stored value $a^2 + b^2 \approx 2\,000\,199.96$ and $2ab \approx 2\,000\,199.95$ both round to 2 000 200, and the identity returns exactly 0. Subtracting first, $(a - b)^2 = 0.09998^2 = 0.009995$, is close to the true 0.01. Subtracting a common centre first (e.g. the training mean), which leaves distances unchanged, removes the problem; so does double precision.

In `numpy`, SqDist is the function below; Note 5, Sections 1–2, builds the nearest-neighbour algorithm on it.

```
def sq_distances(Q, X):                      # Q: (q, d), X: (n, d)
    D = (Q**2).sum(1)[:, None] + (X**2).sum(1) - 2 * Q @ X.T
    return np.maximum(D, 0)                  # round-off can give tiny negatives
```

> **Summary** A dataset is n i.i.d. pairs $(\mathbf{x}, y)$, $\mathbf{x} \in \mathbb{R}^d$, from an unknown $P(\mathbf{X}, Y)$, stored as an $n \times d$ design matrix. The representation — raw values, features, one-hot codes, bags of words, embeddings — fixes what can be learned. Each task has a natural target and loss (Table 2): the conditional mean for squared error, the most probable class for classification error; the remaining Bayes error no algorithm can beat. Everything fitted is fitted on training data only. Pairwise distances cost $O(qnd)$, computed as one matrix product.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. A lookup table stores every training input with its label and otherwise predicts the most frequent class. It has zero training error on data without duplicates; why is it nevertheless a poor learner for handwritten digits? For which kind of input space would it be perfectly reasonable?

A2. For each problem, name the task — classification, regression, density estimation, another unsupervised task, or self-supervised learning — with the target and a natural loss: (a) estimating the redshift of a galaxy from five colours; (b) deciding whether a jet comes from a b quark; (c) flagging collision events that look unlike anything in the training data; (d) grouping light curves of variable stars without labels; (e) predicting the next token of a physics paper.

A3. Is the i.i.d. assumption reasonable for a random train/test split of (a) collision events recorded in one run of an experiment, (b) consecutive frames of the same video, (c) records from 2023–2025 of a detector whose calibration drifts, for a model that will be used in 2026, (d) galaxy images where each galaxy was photographed several times? For each "no", propose a better split.

A4. Give two sentences with identical bag-of-words vectors and different meanings. Which kind of representation could tell them apart, and why?

A5. A feature "detector region" takes the values barrel, end-cap and forward. What goes wrong if

it is coded as $1, 2, 3$ before computing Euclidean distances? How does one-hot encoding fix it, and what is then the distance between two different regions?

A6. In raw pixel space, a vertical bar in a 3×3 image is farther from the same bar shifted by one column than from a horizontal bar. Why does this hurt a method that predicts from nearby training examples? Describing each image by its number of dark pixels, height and width fixes it: what do these features gain, and what do they lose?

A7. True or false, with a one-sentence justification: (a) a sufficiently powerful learning algorithm can reach an expected error (the error rate on the whole population of new examples) below the Bayes error; (b) adding a feature to $\mathbf{x}$ can never increase the Bayes error; (c) if $\mathrm{Var}(Y \mid \mathbf{X} = \mathbf{x}) = 0$ for every $\mathbf{x}$, the best possible mean squared error is zero; (d) the error measured on a finite test set can be below the Bayes error.

A8. A regressor predicts the true energy Y of a particle from the calorimeter reading X; the reading fluctuates by about 2% around the true energy (the resolution of the calorimeter). Interpret the term $\mathrm{Var}(Y \mid X = x)$ of the squared-error decomposition in this setting: which part of the regression error can be reduced by a better algorithm, and which only by a better detector?

A9. Why is cosine similarity preferred to Euclidean distance for bag-of-words vectors of documents of very different lengths? Give two vectors with cosine similarity 1 and a large Euclidean distance.

A10. Identify the leak in each pipeline and say how to remove it: (a) features are standardised with the mean and standard deviation of the full dataset before splitting; (b) the vocabulary of a text classifier is built from training and test documents together; (c) a classifier separating simulated signal from simulated background uses the simulation run number as a feature.

A11. Squared distances from q queries to n points in $\mathbb{R}^d$ can be computed by a triple loop or by one matrix product QX^{T} plus squared norms; both need $O(qnd)$ operations. Why is the second hundreds of times faster, and what limits it when q and n are both very large?

A12. Why is next-token prediction a classification problem? What are m and the loss, and why does a one-hot representation of tokens never need to be written down explicitly?

A13. An event contains between 1 and 12 jets. Compare two fixed-length representations: (i) the $(p_{\mathrm{T}}, \eta, \phi)$ of the four leading jets, padded with zeros, and (ii) five summary observables (multiplicity, H_{T}, the two leading p_{T}, the leading-pair mass). What does each lose? Why can the zeros of padding mislead a method based on Euclidean distances?

B. Short calculations

B1. What is the input dimension d of (a) a 64×64 colour image, (b) a one-hot encoding of a word from a vocabulary of 50 000 words, (c) a sequence of 128 tokens, each embedded in $\mathbb{R}^{768}$, flattened into one vector? How many megabytes (10^6 bytes) does one example of (c) take in single precision?

B2. For $\mathbf{a} = (2, -1, 2)$ and $\mathbf{b} = (1, 4, 8)$ compute $\|\mathbf{a}\|_1$, $\|\mathbf{a}\|_2$, $\|\mathbf{a}\|_\infty$, $\mathbf{a}^{\mathsf{T}}\mathbf{b}$, $\cos(\mathbf{a}, \mathbf{b})$ and $\|\mathbf{a} - \mathbf{b}\|^2$, the last one in two ways.

B3. The points $\mathbf{x}_1 = (0, 1)$, $\mathbf{x}_2 = (3, 1)$ and $\mathbf{x}_3 = (1, 3)$ form the rows of X. Compute the vector of squared norms, the Gram matrix XX^{T} and, using $\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a}^{\mathsf{T}}\mathbf{b}$, the matrix of squared distances. Check one entry directly; give each point's nearest neighbour.

B4. Build the vocabulary and bag-of-words vectors for the training documents "proton hits target", "neutron hits target" and "proton proton collision". Encode the new document "neutron hits proton" and compute its cosine similarity with each training document. Rank the training documents by similarity. Two of them tie: explain from the word counts why, and why a bag of words cannot break the tie.

B5. The joint distribution of a binary feature $X \in \{0,1\}$ and a label $Y \in \{0,1\}$ is $P(0,0) = 0.35$, $P(0,1) = 0.15$, $P(1,0) = 0.10$, $P(1,1) = 0.40$. Compute $P(Y = 1 \mid X = x)$ for both x, the Bayes classifier and the Bayes error. What is the error of the best classifier that ignores X?

B6. Given $X = x$, a target Y takes the values 0, 2 and 7 with probabilities 0.5, 0.3 and 0.2. Write $g(c) = \mathbb{E}[(c - Y)^2 \mid X = x]$ as a quadratic in c, find its minimum by setting $g'(c) = 0$, and check that the minimum value equals $\mathrm{Var}(Y \mid X = x)$. What is the expected squared error of the prediction $c = 3$?

B7. A design matrix has $n = 10^6$ rows and $d = 512$ columns. How much memory does it need in single precision? How many floating-point operations does it take to compute the squared distances from $q = 1000$ queries to all rows (count $2qnd$ for the product), and roughly how long on a machine sustaining 10^{11} operations per second?

C. Derivations

C1. Without looking at Section 4.1, reproduce (1) and (2). Apply the latter to $f \equiv \mathbb{E}[Y]$ to prove $\mathrm{Var}(Y) = \mathbb{E}[\mathrm{Var}(Y \mid \mathbf{X})] + \mathrm{Var}(\mathbb{E}[Y \mid \mathbf{X}])$, and check it on the regression example of Section 4.1.

C2. Show that predicting class c at $\mathbf{x}$ has conditional error $1 - \eta_c(\mathbf{x})$, so that the Bayes classifier is (3). Then, starting from $R^* = \mathbb{E}\big[1 - \max_k \eta_k(\mathbf{X})\big]$, show that the Bayes error is at most $1 - \frac{1}{m}$ for m classes (hint: the largest of m numbers that add up to 1 is at least $\frac{1}{m}$), and describe a distribution of $(\mathbf{X}, Y)$ for which the bound is attained.

C3. For the absolute loss $|c - Y|$ and a continuous conditional density of Y, show that $\frac{d}{dc}\mathbb{E}[\,|c - Y| \mid \mathbf{X} = \mathbf{x}] = P(Y < c \mid \mathbf{X} = \mathbf{x}) - P(Y > c \mid \mathbf{X} = \mathbf{x})$, and conclude that the best prediction is the conditional *median*. Why is the median less sensitive than the mean to rare, very large values of Y?

C4. Prove the Cauchy–Schwarz inequality by noting that $h(t) = \|\mathbf{a} - t\mathbf{b}\|^2 \geq 0$ for every real t: expand h as a quadratic in t and use its minimum. Conclude that $-1 \leq \cos(\mathbf{a}, \mathbf{b}) \leq 1$ and find when equality holds.

C5. A dataset contains G groups of r near-identical examples each, and a random split puts a fraction τ of the $N = Gr$ examples in the test set. Show that the probability that a test example has none of its $r - 1$ siblings in the training set is $\binom{\tau N - 1}{r-1} / \binom{N-1}{r-1}$, and that this is approximately τ^{r-1} for large N. Evaluate both for the galaxy example.

D. Programming

D1. Implement both functions of Algorithm 5, check that they agree, time them for $q = 1000$, $n = 10\,000$, $d = 128$ (extrapolate the loop), and add a blocked version returning each query's nearest point. Reproduce the `float32` cancellation of Section 6.1; show that centring cures it.

D2. Implement the three functions of Algorithm 2 with Python dictionaries, BagOfWords returning a `scipy.sparse` matrix. Reproduce the cosines of Section 6 with and without stop words,

and measure how dense and sparse memory grow with the number of documents.

D3. Reproduce the feature-selection experiment of Section 5.1: 50 examples, 5000 noise features, selection of 20 features, 5-fold cross-validation of a 1-nearest-neighbour classifier, with the selection done once on all data and, alternatively, inside each fold. Repeat over 200 datasets and report both average errors. How does the gap change with the number of selected features?

Further reading. Bishop [1], ch. 1; Murphy [11], ch. 1; Hastie, Tibshirani and Friedman [7], sections 2.4 and 7.10; for physics, Carleo *et al.* [3].

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[2] T. Brown *et al.*, Language models are few-shot learners, *NeurIPS* (2020).

[3] G. Carleo *et al.*, Machine learning and the physical sciences, *Rev. Mod. Phys.* 91, 045002 (2019).

[4] N. Dalal and B. Triggs, Histograms of oriented gradients for human detection, *CVPR* (2005).

[5] J. Devlin *et al.*, BERT: pre-training of deep bidirectional transformers for language understanding, *NAACL* (2019).

[6] T. Gebru *et al.*, Datasheets for datasets, *Commun. ACM* 64(12), 86–92 (2021).

[7] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[8] S. Kapoor and A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, *Patterns* 4(9), 100804 (2023).

[9] G. Kasieczka *et al.*, The machine learning landscape of top taggers, *SciPost Phys.* 7, 014 (2019).

[10] D. G. Lowe, Distinctive image features from scale-invariant keypoints, *Int. J. Comput. Vis.* 60, 91–110 (2004).

[11] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[12] L. Ouyang *et al.*, Training language models to follow instructions with human feedback, *NeurIPS* (2022).

[13] A. Radford *et al.*, Learning transferable visual models from natural language supervision, *ICML* (2021).

[14] O. Russakovsky *et al.*, ImageNet large scale visual recognition challenge, *Int. J. Comput. Vis.* 115, 211–252 (2015).

[15] A. Vaswani *et al.*, Attention is all you need, *NeurIPS* (2017).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.